Processor Architecture Coding Assignment Writeup

Anna Wu

1 October 2025

0.1 Overall

As far as I can tell, my code is in a mostly functional state. In addition to the `BEQ` test that was provided with the base code, I created 10 tests to evaluate most of the mini-mips instructions in one way or another. For the scope of the assignment, I decided not to test every single type of instruction (though all are implemented, likely to varying degrees of success) as that seemed excessive. Instead, I tested all of the operations I see most frequently in our class examples and decided that for instructions with a nearly identical flow I would accept the passed test as sufficient for all those instructions (e.g. I do not test `XOR` individually, but the `instruction` struct that the `decode` function outputs is nearly identical to the `OR` operation save for the actual `op` field, so for the purposes of how I felt about my code I decided a passed `OR` test was enough to have confidence in both).

0.2 Testing

I directly tested all three jumps (jal, jump, and jr), instructions that use `HI` and `LO` (mult and div), memory instructions (store and load), simple arithmetic (add, addi, sub), and branch (if equal). Through these, some of those instructions end up doing `OR` and `SLL` operations, so for me that was enough to feel confident in the basic logical and shifting operations. I am certain that some of the more specific shifting operations are wrong, namely that I don't differentiate between the shift by `shamt` versus `rt` instructions, and I think that only if it is a jump does it actually shift by the `shamt` in the way I have it set up. I didn't go into testing this exactly for the sake of having already spent far too many hours on this while in the midst of thesising, so I can't be certain but that is my guess as to what has the most unintended/unpredictable behavior in my code.

For testing, I found that I couldn't tell if a failed outcome was a result of writing the test incorrectly or something actually wrong with my implementation, so I added more debugging statements to the `run_one_instruction` function. This is why I have an extra file, `stage_test.h`, which exports a few simple functions to convert the op enum cases to strings for debug logging. Now, each instruction prints its return value at each stage of the architecture. For instance, a simple `ADD` instruction where we add register 2 and register 3 will output the following in the console:

```
------RUNNING TEST: add------
FETCH: PC = 0, instruction = 0x00432020
DECODE: op=ADD, ctrl=WB, left=2, right=-3, dest=4, extra=0
EXECUTE: value=-1, hi_val=0, dest=4, op=WRITEBACK, jump=0
MEMORY: value=-1, dest=4, op=WRITEBACK
WRITEBACK: reg 4 = -1
PC incremented to 4
```

Of the 10 tests that I wrote plus the given `BEQ` test, all 11 are passing.

0.3 Weirdness

In terms of some of the more ambiguous solutions in my code, there are various spots where I wasn't sure exactly the intended way of handling some operations. For instance, in handling `BRANCH` operations in the `execute` function, I assume the ALU should handle the equality check (this will be either `BEQ` or `BNE` depending on the decode), and then I manually shift the address 2 to the right if the branch is equal as though I am using the XTND hardware. Similarly, in the `JAL` operation, I manually compute pc + 4 to store that in $31 (assuming that would be like using the +4 hardware) because the ALU is busy doing the shifting in that one. Additionally, I do a sort of weird thing in the `memory` function for STORE, where (based on how I set things up earlier) the `dest` property ends up holding the value we want to write to memory and `value` has the address that we want to write to. I do it differently though in the jump operations, which you can see in the `run_one_instruction` function where I update the PC, because there I store the address to jump to in `hi_val`. Also, something about my print statements for the errors if an instruction has an invalid format is a little weird and it won't actually say which register/argument is wrong. With the rest of my aforementioned debug code though, you can tell despite that confusion. I believe I've left comments in my code about these assumptions in all the places I noted them.